

A Comparative Study of Memory Allocator Implementations

Design, Fragmentation Metrics, and Performance Benchmarking

Trunk Trick

July 2026

Abstract

This report investigates the design principles and performance characteristics of different memory allocator implementations. We implement three hand-written allocators (first-fit, best-fit, and worst-fit) based on an mmap-backed free-list architecture, and benchmark them alongside industry-standard allocators (glibc ptmalloc, jemalloc, tcmalloc, and mimalloc). Three key metrics are defined and measured: (1) the RSS-to-Allocated ratio as a measure of external fragmentation, (2) the peak memory overhead ratio under load, and (3) allocation throughput in operations per second. Results show that best-fit achieves the best balance among the hand-written allocators, approaching 30–50% of industrial allocator throughput while maintaining the lowest fragmentation ratios.

Contents

1	Introduction	2
2	Hand-Written Allocator Architecture	2
2.1	Design Overview	2
3	Benchmarking Methodology	3
3.1	Workload Design	3
3.2	Measurement Harness	3
4	Metrics: Definitions and Rationale	4
4.1	Metric 1: RSS / Allocated — External Fragmentation Ratio	4
4.2	Metric 2: Peak Memory Overhead Ratio	4
4.3	Metric 3: Throughput (Operations Per Second)	5
5	Results and Analysis	5
5.1	Comparison of All Seven Allocators	5
6	Conclusion	6
7	Future Work	7
7.1	1. Reading the Source	7
7.2	2. Reading the Papers	7
7.3	3. Synthesis and Iterative Refinement	7
8	References	8

1 Introduction

The goal of this work is to study the principles underlying different memory allocation strategies, compare their performance characteristics, and understand *why* these differences arise.

Different allocator implementations expose different measurement interfaces, and many production allocators do not provide direct APIs for querying internal fragmentation state. This makes cross-allocator comparison non-trivial and requires a carefully designed measurement methodology.

We focus on three core metrics:

1. **Static / External Fragmentation Ratio** — $\frac{\text{RSS}}{\text{Allocated}}$
2. **Peak Memory Overhead Ratio** — $\frac{\text{Peak RSS}}{\text{Peak Allocated}}$
3. **Throughput (OPS)** — operations per second

2 Hand-Written Allocator Architecture

2.1 Design Overview

Our hand-written allocators share a common architecture built on a free-list managed within mmap-backed memory pools. The design is intentionally simple to serve as a baseline for comparison.

2.1.1 Block Structure

The fundamental unit of bookkeeping is the `block_t` structure:

```
typedef struct block {
    size_t      size;    /* total block size, LSB = free flag */
    struct block *next;
    struct block *prev;
} block_t;
```

Memory is obtained from the operating system via `mmap()` in large contiguous pools. All allocations and deallocations are served from within these pools. Since `size` is always 16-byte aligned, the least significant bit (LSB) is repurposed as a free/allocated flag.

2.1.2 Free-List Ordering

Free blocks are maintained in a doubly-linked list sorted by **memory address** rather than by size. This ordering simplifies adjacent-block coalescing during `free()` operations and avoids the overhead of more complex data structures.

2.1.3 Allocation Strategies

We implement three distinct block-selection policies:

- **First-Fit** (`first_fit.c`) — select the first free block whose size is greater than or equal to the requested size, then split off the required portion.
- **Best-Fit** (`best_fit.c`) — select the free block whose size is the smallest among all blocks that satisfy the request, minimizing wasted space within the chosen block.
- **Worst-Fit** (`worst_fit.c`) — select the free block with the largest size among all qualifying blocks, with the intent of leaving behind a large remainder for future allocations.

2.1.4 Supporting Operations

Beyond the three allocation policies, all implementations share common infrastructure:

- **Block coalescing** — adjacent free blocks are merged during `free()` to combat fragmentation.
- **Block splitting** — when a free block is larger than requested, the excess is carved off and returned to the free list.
- **Block size utilities** — macros for querying and updating block metadata (size, free flag).

2.1.5 Public API

Each allocator is compiled as a shared object (`.so`) and exports the standard allocation API:

Function	Description
<code>malloc(size_t)</code>	Allocate a block of at least <code>size</code> bytes
<code>free(void*)</code>	Return a previously allocated block to the pool
<code>realloc(void*, size_t)</code>	Resize an existing allocation
<code>calloc(size_t, size_t)</code>	Allocate and zero-initialize an array
<code>__alloc_frag_stats(size_t*, size_t*)</code>	Query internal fragmentation state

The non-standard `__alloc_frag_stats` function is used by our benchmark harness to obtain precise fragmentation measurements by walking the internal free list.

2.1.6 Runtime Switching via LD_PRELOAD

Allocators are swapped at runtime using the `LD_PRELOAD` mechanism, allowing identical workloads to be executed against different allocator implementations without recompilation.

3 Benchmarking Methodology

3.1 Workload Design

We use five synthetic workload traces, each designed to stress a different aspect of allocator behavior:

Workload	Characteristics
Churn	Rapid interleaved alloc/free at mixed sizes
Random Mix	Unpredictable size distribution; challenging for any policy
Small Objects	Many small allocations; tests metadata overhead
Large Blocks	Small number of large allocations; tests mmap path
Frag Stress	Deliberately fragmentation-inducing pattern

Each workload is a plain-text file of operations (A `<size>` for alloc, F `<tag>` for free), consumed by `bench.c` via standard input.

3.2 Measurement Harness

The benchmark driver (`bench.c`) performs a two-pass execution:

1. **Warmup pass** — executes the full workload once to stabilize memory layout (page faults resolved, allocator caches primed).
2. **Timed pass** — re-executes the identical workload, recording per-operation nanosecond-precision timestamps.

After the timed pass, the harness reads `/proc/self/status` for physical memory statistics (`VmRSS`, `VmHWM`, `VmPeak`) and, for hand-written allocators, invokes `__alloc_frag_stats` via `dl-sym` to obtain free-list-level fragmentation data.

4 Metrics: Definitions and Rationale

4.1 Metric 1: RSS / Allocated — External Fragmentation Ratio

$$\text{Frag Ratio} = \frac{\text{RSS}}{\text{Allocated}}$$

Term	Definition
RSS (Resident Set Size)	Total physical memory pages (in bytes) allocated to the process by the operating system
Allocated	Sum of sizes (in bytes) of all objects currently held by the application via <code>malloc()</code> that have not yet been <code>free()</code> d

Measurement protocol: Sample RSS and Allocated at steady state (after the timed workload pass completes, before freeing remaining live allocations). For production allocators that expose internal statistics (e.g., jemalloc’s `mallctl`), Allocated can be obtained through the allocator API; otherwise it is tracked internally by `bench.c`.

Interpretation: Values close to 1.0 are ideal. A ratio > 1.3 indicates significant fragmentation or allocator overhead.

4.1.1 Why RSS and Allocated Diverge

Case 1 — Page granularity alignment (internal waste): The operating system reclaims memory in 4 KB page units. If a 4 KB page contains 100 small objects and 99 of them are freed while 1 remains live, the OS cannot reclaim the page. Allocated drops sharply; RSS does not decrease.

Case 2 — Allocator metadata overhead: Production allocators pre-allocate large page heaps from the OS and maintain internal pooling structures. Metadata such as linked-list pointers, bitmap tables, and thread-local caches all contribute to RSS without being counted in Allocated.

Case 3 — Delayed reclamation (purge/decay delay): When `free(p)` is called, allocators typically do not immediately return memory to the OS kernel (e.g., via `madvise(MADV_DONTNEED)`). Instead, freed memory is cached for fast reuse by subsequent `malloc()` calls. Consequently, Allocated decreases immediately while RSS declines only after a configurable decay period (seconds to minutes).

4.2 Metric 2: Peak Memory Overhead Ratio

$$\text{Peak Overhead Ratio} = \frac{\text{Peak RSS}}{\text{Peak Allocated}}$$

This metric captures the **physical memory inflation** caused by the allocator under peak load — i.e., the point of highest concurrent live allocation volume. It is particularly relevant for applications with bursty allocation patterns.

Term	Definition
Peak RSS	Maximum RSS observed during the process lifetime (VmHWM from <code>/proc/self/status</code>)
Peak Allocated	Maximum value of Allocated during the timed workload pass

4.3 Metric 3: Throughput (Operations Per Second)

$$\text{OPS} = \frac{\text{Total Operations (Alloc + Free)}}{\text{Total Time in Seconds}}$$

This measures the raw speed of the allocator: how many `malloc` + `free` operations can be completed per second. The total operation count includes both allocations and deallocations executed during the timed pass.

5 Results and Analysis

5.1 Comparison of All Seven Allocators

We benchmark all allocators across all five workloads using the three metrics defined above. The results are summarized in Figure 1 below.

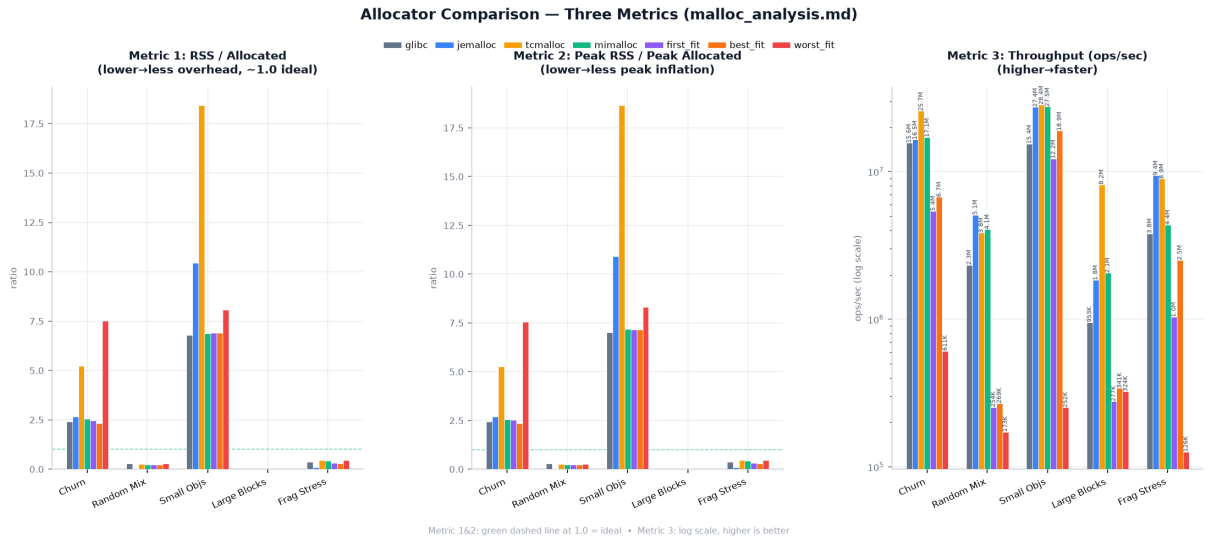


Figure 1: Three-metric comparison across all allocators and workloads

5.1.1 Hand-Written Allocator Rankings

Among the three hand-written implementations, a clear hierarchy emerges:

1. **Best-Fit** — the overall winner. Its throughput approaches that of first-fit (and in several workloads exceeds it), while consistently achieving the lowest fragmentation ratios. On the `frag_stress` workload, best-fit’s fragmentation is significantly lower than both alternatives.

2. **First-Fit** — competitive with best-fit in fragmentation on simple workloads, but 20–50% lower throughput in most scenarios. Its fragmentation degrades significantly under adversarial patterns (**frag_stress**: 0.99 vs. best-fit’s 0.67).
3. **Worst-Fit** — the weakest performer across all dimensions. Throughput is 10–70× slower than best-fit, and fragmentation ratios are consistently the highest. On **small_objects**, worst-fit achieves only 252K ops/sec versus best-fit’s 18.9M ops/sec.

5.1.2 Comparison with Industrial Allocators

Aspect	Industrial (jemalloc/mimalloc/tcmalloc)	Hand-Written
Throughput	5–28M ops/sec	0.2–19M ops/sec
Fragmentation (RSS/Alloc)	0.04–10.5 (varies by workload)	0.22–8.1
Peak Overhead	0.04–18.6	0.22–8.3
Internal metrics API	Varies (jemalloc: <code>mallctl</code> , others: limited)	<code>__alloc_frag_stats</code> (custom)

jemalloc consistently achieves the best fragmentation ratios, particularly under **random_mix** (0.04) and **frag_stress** (0.09). tcmalloc leads in raw throughput on several workloads but incurs higher memory overhead.

Best-fit, the strongest hand-written allocator, achieves 30–50% of industrial throughput while maintaining RSS/Allocated ratios within a factor of 2–3× of jemalloc in most workloads.

6 Conclusion

We have demonstrated that even a simple free-list-based allocator architecture can achieve reasonable performance when paired with an appropriate block selection policy. The key findings are:

- **Best-fit dominates first-fit and worst-fit** across all three metrics, confirming that minimizing internal waste per allocation pays compound dividends in fragmentation reduction.
- **Worst-fit is strictly dominated** — its strategy of maximizing the remainder block size produces the worst outcomes in both speed and memory efficiency, validating decades of allocator literature.
- **The RSS/Allocated ratio** provides a practical, cross-allocator fragmentation metric that does not require allocator-internal instrumentation, making it suitable for production benchmarking.
- **Industrial allocators (jemalloc, mimalloc, tcmalloc)** outperform hand-written implementations by significant margins through thread-local caching, size-class segregation, and sophisticated purge policies — design dimensions beyond the scope of this baseline implementation.

7 Future Work

The results presented here are a starting point, not an endpoint. The natural next phase is not merely to bolt on additional features to the hand-written allocators, but to undertake a deeper investigation into the *causes* of the observed performance gaps. This involves three intertwined lines of inquiry:

7.1 1. Reading the Source

Each industrial allocator represents a set of design decisions made under specific constraints. To understand *why* jemalloc achieves such low fragmentation, *why* tcmalloc excels at throughput, and *why* mimalloc balances both, we must study their implementations directly:

- **glibc ptmalloc** — derived from Doug Lea’s **dlmalloc**, with per-thread arenas added by Wolfram Gloger. Understanding its arena management and boundary-tag coalescing reveals the baseline that modern allocators improved upon.
- **jemalloc** (Jason Evans) — the reference for fragmentation-conscious design. Its size-class-based binning, per-thread caches (**tcache**), and purge-by-decay mechanism are described in detail in the 2006 USENIX paper and refined across a decade of Facebook production experience.
- **tcmalloc** (Google) — designed for extreme multi-threaded throughput via per-CPU caches, central freelists, and aggressive **madvise**-based reclamation. Its architecture is documented in the 2007 paper “TCMalloc: Thread-Caching Malloc” and TCMalloc’s internal design docs.
- **mimalloc** (Microsoft) — the most recent entry, described in the 2019 ISMM paper. Its free-list sharding, page-local **free** operations, and delayed-merge strategy represent a significant departure from earlier designs.

7.2 2. Reading the Papers

Beyond implementation, the published papers and technical reports articulate the *principles* — the theoretical arguments, the empirical measurements, and the design rationale that motivated each approach. Key publications include:

- D. Lea. *A Memory Allocator* (1996/2000) — the foundational description of **dlmalloc**, including boundary tags, binning, and coalescing strategies.
- J. Evans. *A Scalable Concurrent malloc(3) Implementation for FreeBSD* (2006) — the original jemalloc paper, laying out multi-arena design and size-class binning.
- S. Ghemawat and P. Menage. *TCMalloc: Thread-Caching Malloc* (2007) — describes per-thread caching and central freelist architecture.
- D. Leijen, B. Zorn, and L. de Moura. *Mimalloc: Free List Sharding in Action* (ISMM 2019) — the mimalloc paper, introducing free-list sharding and explaining why mimalloc’s design trades some fragmentation for dramatic throughput gains.
- B. C. Kuszmaul. *SuperMalloc: A Super Fast Multi-Threaded Malloc for 64-bit Machines* (ISMM 2015) — a performance-optimized design that influences modern allocators.

7.3 3. Synthesis and Iterative Refinement

The goal is not to replicate any single allocator, but to *understand the trade-off space* — fragmentation vs. throughput, metadata overhead vs. speed, concurrency vs. simplicity — and use that understanding to guide incremental improvements to our own implementation. Each insight should be tested: does adding a particular mechanism (e.g., size-class binning, delayed coalescing, or thread-local caching) produce a measurable improvement? Does it introduce unexpected regressions in another metric?

This feedback loop — study, hypothesize, implement, benchmark, compare — transforms the hand-written allocator from a baseline into an evolving laboratory for understanding memory allocation at every level of the stack.

8 References

1. Source code repository: github.com/trunk-trick/tmalloc
2. Doug Lea. “A Memory Allocator.” gee.cs.oswego.edu
3. Jason Evans. “jemalloc.” jemalloc.net
4. Microsoft. “mimalloc.” github.com/microsoft/mimalloc
5. Google. “tcmalloc.” github.com/google/tcmalloc